

HORYZONS GROUP
DOCUMENT INTERNE — CONFIDENTIEL

STATIA

Plateforme IDE d'analyse de données pilotée par intention naturelle

CAHIER DES CHARGES & CATALOGUE SCIENTIFIQUE

Version 1.0
Togo — Lomé
Rédigé pour Daven — Fondateur, Horyzons Group

Table des matières

1. Résumé exécutif

STATIA est une plateforme d'analyse de données nouvelle génération, pensée comme un « IDE de la donnée » : l'utilisateur importe un ou plusieurs fichiers (CSV, Excel, bases SQL), décrit en langage naturel ce qu'il souhaite comprendre, et le système construit automatiquement le plan d'analyse statistique, exécute les calculs via un moteur mathématique déterministe et vérifié, puis restitue les résultats sous forme de graphiques, cartographies, tableaux de bord interactifs et explications rédigées.

Le positionnement est double : offrir la puissance combinée d'un logiciel statistique professionnel (R, SPSS, MATLAB Statistics Toolbox) et d'un outil de Business Intelligence (Power BI, Tableau), tout en supprimant la barrière technique qui oblige aujourd'hui l'utilisateur à connaître le nom exact des fonctions, la syntaxe du langage et les conditions d'application des méthodes.

Le principe non négociable de l'architecture est la séparation stricte entre l'intelligence artificielle, qui joue le rôle d'analyste et d'orchestrateur, et le moteur de calcul, qui seul a l'autorité d'exécuter et de vérifier les opérations mathématiques. L'IA ne calcule jamais un résultat statistique elle-même : elle comprend l'intention, sélectionne les méthodes appropriées dans un catalogue de formules documentées, déclenche leur exécution réelle, vérifie que les conditions d'application sont respectées, et traduit le résultat en langage clair.

« L'utilisateur connaît sa question. Il fournit ses données. L'intelligence artificielle comprend l'intention et construit le plan d'analyse.

Le moteur mathématique exécute et valide. La plateforme restitue le résultat, le graphique et l'explication — jamais une boîte noire. »

1.1 Objectifs du produit

- Permettre à un utilisateur non technique d'obtenir une analyse statistique rigoureuse à partir d'une simple description en langage naturel.
- Garantir l'exactitude et la reproductibilité de chaque résultat par un moteur de calcul déterministe, indépendant du modèle de langage.
- Offrir une couche de visualisation et de tableaux de bord comparable aux standards de la Business Intelligence (Power BI-like), incluant la cartographie géographique ciblée.
- Construire un catalogue exhaustif, documenté et extensible de méthodes mathématiques, probabilistes et statistiques, chacune assortie de ses conditions d'application, ses limites et ses formules exactes.
- Fournir une fonctionnalité de transparence (« Show me why ») qui explique, à la demande, pourquoi telle méthode a été choisie plutôt qu'une autre.
- Construire une architecture technique modulaire, en microservices, mêlant Rust (performance, fiabilité, traitement de données volumineuses) et Python (écosystème scientifique, orchestration IA).

1.2 Ce que STATIA n'est pas

- Ce n'est pas un générateur de texte qui « invente » des statistiques : chaque chiffre affiché provient d'un calcul réellement exécuté et vérifié.

- Ce n'est pas un simple assistant conversationnel branché sur R ou Python en coulisses sans contrôle : chaque appel est tracé, validé et explicable.
- Ce n'est pas, dans sa version 1, un concurrent complet de Power BI : la couche visualisation avancée (dashboards riches, cartographie) est un chantier en soi, construit après la fiabilisation du moteur d'analyse.

2. Vision, positionnement et utilisateurs cibles

2.1 Vision produit

Aujourd'hui, l'analyse de données sérieuse exige de savoir quelle méthode utiliser (test t, ANOVA, régression logistique...), de connaître la syntaxe d'un langage (R, Python, SQL), et de vérifier soi-même que les conditions mathématiques d'application sont réunies (normalité, homoscédasticité, indépendance des observations, etc.). Cette barrière exclut la grande majorité des porteurs de projets, entrepreneurs, chercheurs juniors et décideurs qui ont des données mais pas le bagage statistique ou technique pour les exploiter pleinement.

STATIA inverse ce paradigme. L'utilisateur exprime son besoin analytique dans son propre langage — français ou anglais — et le système se charge de traduire cette intention en une démarche scientifique rigoureuse, reproductible et vérifiable. La plateforme se positionne ainsi comme un environnement de « vibe coding » appliqué à la donnée : de la même manière qu'un outil comme Cursor permet de décrire une fonctionnalité logicielle en langage naturel et d'obtenir du code fonctionnel, STATIA permet de décrire une question analytique et d'obtenir une analyse statistique complète, vérifiée et documentée.

2.2 Utilisateurs cibles

Catégorie	Éléments couverts	Usage cible dans la plateforme
Entrepreneurs & PME	Propriétaires de commerces, gérants de PME sans compétence data	Comprendre leurs ventes, leur clientèle, leurs tendances sans recruter un analyste
Chercheurs & étudiants	Mémoires, thèses, projets académiques	Obtenir des analyses statistiquement défendables avec justification méthodologique
Institutions & ONG	Enquêtes de terrain, données socio-économiques (UEMOA, Afrique de l'Ouest)	Cartographie ciblée, tableaux de bord de suivi, rapports d'impact
Analystes & Data teams	Utilisateurs techniques cherchant à accélérer leur travail	Prototypage rapide d'analyses avant industrialisation en R/Python

2.3 Proposition de valeur différenciante

- Le moteur ne devine jamais : chaque méthode statistique appliquée est sélectionnée sur la base de critères documentés (type de variable, taille d'échantillon, normalité, etc.) et ce raisonnement est consultable par l'utilisateur.
- La plateforme est pensée pour le contexte ouest-africain dès sa conception : gestion multilingue (français prioritaire), cartographie adaptée aux découpages administratifs locaux, port vers des cas d'usage mobile money, agriculture, santé publique, éducation.
- L'architecture technique (Rust + Python en microservices) est conçue pour tenir la charge de gros volumes de données dès la conception, contrairement à des prototypes construits uniquement en notebook.

3. Périmètre fonctionnel détaillé

3.1 Ingestion et préparation des données

- Import de fichiers CSV, TSV, XLSX, XLS, JSON, Parquet.
- Connexion à des bases de données relationnelles (PostgreSQL, MySQL) — phase 2.
- Détection automatique des types de colonnes (numérique, catégoriel, date, texte, géographique, identifiant).
- Détection et traitement des valeurs manquantes (suppression, imputation par moyenne/médiane/mode, imputation multiple).
- Détection des valeurs aberrantes (outliers) par méthodes IQR, Z-score, Isolation Forest.
- Normalisation, standardisation, encodage des variables catégorielles (one-hot, ordinal, target encoding).
- Fusion de plusieurs fichiers/jeux de données (jointures, concaténation).
- Aperçu automatique du jeu de données : nombre de lignes/colonnes, pourcentage de valeurs manquantes, distribution sommaire de chaque variable.

3.2 Interaction en langage naturel (couche « vibe analysis »)

- Zone de prompt principale acceptant une description libre de l'objectif analytique.
- Compréhension du prompt et extraction structurée : objectif, variable(s) cible(s), variables explicatives, type de problème (description, comparaison, association, prédiction, segmentation, série temporelle).
- Dialogue de clarification automatique si le prompt est ambigu (ex. variable cible non identifiable, colonnes homonymes).
- Historique conversationnel permettant d'affiner une analyse déjà lancée (« et si on excluait les valeurs de 2023 ? »).
- Fonction « Show me why » : explique, sur demande, les critères ayant conduit au choix d'une méthode plutôt qu'une autre.

3.3 Moteur de planification analytique

- Construction automatique d'un plan d'analyse structuré (JSON) à partir de l'intention détectée.
- Sélection de la méthode statistique appropriée selon un arbre de décision documenté (type de variable, nombre de groupes, taille d'échantillon, normalité, indépendance).
- Vérification préalable des hypothèses (normalité, homoscedasticité, indépendance, multicolinéarité) avant l'exécution d'un test ou d'un modèle.
- Bascule automatique vers une méthode alternative (ex. test non paramétrique) si les conditions d'application ne sont pas réunies, avec notification claire à l'utilisateur.

3.4 Moteur de calcul statistique et mathématique

Détaillé intégralement dans la section 5 (catalogue mathématique et statistique complet). Le moteur expose chaque fonction sous forme de service versionné, documenté et testé unitairement, avec une fiche technique standardisée : entrées, sorties, hypothèses, méthode numérique, limites, sources de référence.

3.5 Visualisation et restitution (couche « Power BI-like »)

- Génération automatique de graphiques adaptés au type de résultat : histogrammes, boîtes à moustaches, nuages de points, courbes, diagrammes en barres, diagrammes circulaires, heatmaps de corrélation, graphiques de résidus.
- Tableaux de bord interactifs personnalisables : glisser-déposer de widgets, filtres croisés, segmentation dynamique (« slicers »).
- Cartographie ciblée : cartes choroplèthes par région/préfecture/commune, cartes de points géolocalisés, cartes de densité (heatmap géographique), agrégation par zone administrative personnalisée.
- Export des résultats et dashboards en PDF, PNG, XLSX, PowerPoint.
- Partage de tableaux de bord via lien sécurisé, avec gestion de droits (lecture seule / édition).

3.6 Explicabilité, traçabilité et confiance

- Journal complet de chaque étape du pipeline : détection → planification → exécution → validation → visualisation.
- Affichage systématique du niveau de confiance, de la p-value, des intervalles de confiance et des limites de l'analyse.
- Reproductibilité : chaque analyse peut être rejouée à l'identique et exportée sous forme de script (Python) pour un utilisateur avancé souhaitant l'auditer ou la faire évoluer manuellement.

3.7 Modes d'usage progressifs

Catégorie	Éléments couverts	Usage cible dans la plateforme
Mode Simple	Calculs directs et descriptifs (« quelle est la moyenne des ventes ? »)	Utilisateurs non techniques, besoin immédiat
Mode Analyste	Comparaisons et tests automatiques entre groupes/variables	PME, gestionnaires, chargés d'étude
Mode Avancé	Modélisation complète, sélection de variables, diagnostics	Analystes, chargés de recherche
Mode Recherche	Inférence rigoureuse, tests d'hypothèses formalisés, bayésien	Chercheurs, doctorants, publications scientifiques

4. Architecture technique

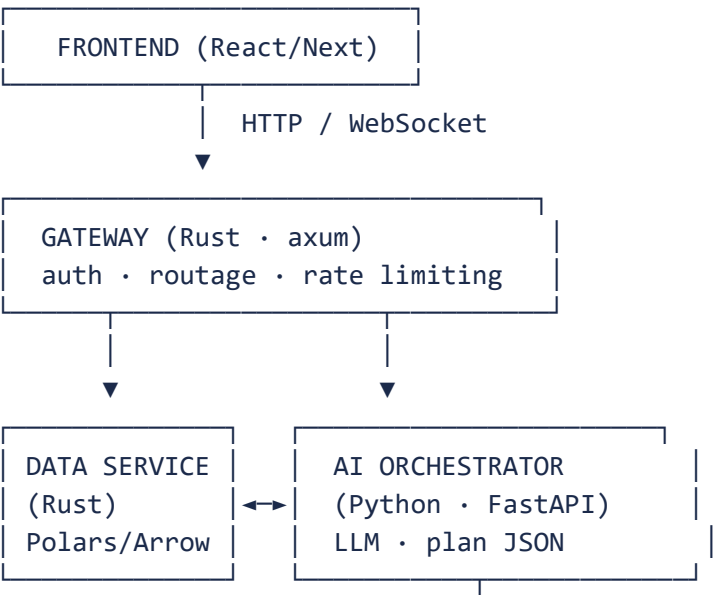
4.1 Principe directeur

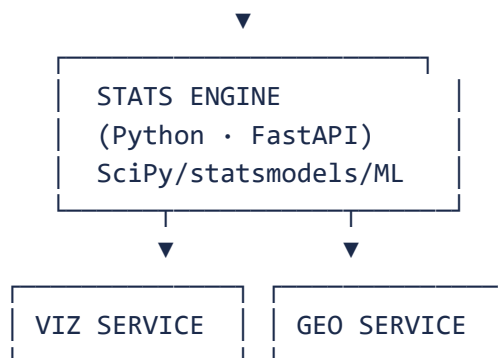
L'architecture repose sur trois systèmes indépendants communiquant entre eux : un moteur de calcul, un orchestrateur d'intelligence artificielle, et une interface utilisateur. Rust est utilisé pour tout ce qui exige performance, fiabilité et traitement sûr de volumes de données importants (ingestion, parsing, passerelle API). Python est utilisé pour tout ce qui relève de l'écosystème scientifique et de l'orchestration du modèle de langage, domaine où ses bibliothèques (SciPy, statsmodels, scikit-learn, pandas, Polars) sont inégalées en maturité.

4.2 Répartition des responsabilités par service

Catégorie	Éléments couverts	Usage cible dans la plateforme
Gateway	Rust (axum) — authentification, routage, limitation de débit, upload de fichiers volumineux en flux	Point d'entrée unique de la plateforme
Data Service	Rust (Polars/Arrow) — parsing CSV/XLSX, typage, nettoyage, stockage temporaire en mémoire	Transformation du fichier brut en jeu de données structuré et validé
AI Orchestrator	Python (FastAPI) — compréhension du prompt, construction du plan d'analyse JSON, appel LLM, boucle de vérification	Traduction de l'intention utilisateur en plan d'exécution
Stats Engine	Python (FastAPI) — SciPy, statsmodels, scikit-learn ; registre de formules documentées	Exécution réelle et vérifiée des calculs statistiques
Viz Service	Python — génération de spécifications graphiques (Vega-Lite / Plotly)	Production des graphiques et éléments de tableaux de bord
Geo Service	Python/Rust — traitement géospatial, projections, agrégation par zone administrative	Cartographie ciblée et analyses géographiques

4.3 Schéma d'architecture





4.4 Communication inter-services

- Phase de démarrage : HTTP/JSON entre tous les services, pour privilégier la vitesse d'itération et la simplicité de débogage.
- Phase de consolidation : migration du chemin critique (Data Service ↔ Stats Engine) vers gRPC pour un typage fort et une meilleure performance.
- Format d'échange en mémoire recommandé : Apache Arrow, nativement supporté par Polars (Rust) et par l'écosystème Python (pandas, Polars), afin d'éviter les coûts de sérialisation répétée.

4.5 Structure d'une fiche de méthode dans le registre du moteur

Chaque fonction du Stats Engine est décrite par une fiche standardisée, condition nécessaire pour que l'IA puisse sélectionner une méthode sur la base de critères documentés plutôt que par imitation statistique :

```

{
  "name": "t_test_independent",
  "category": "hypothesis_testing",
  "inputs": { "group_a": "array<float>", "group_b": "array<float>" },
  "assumptions": ["normalité (Shapiro-Wilk)", "égalité des variances (Levene)"],
  "fallback_if_assumptions_fail": "mann_whitney_u",
  "outputs": { "statistic": "float", "p_value": "float", "ci_95": "tuple" },
  "numerical_method": "scipy.stats.ttest_ind",
  "limitations": ["sensible aux outliers", "nécessite indépendance des échantillons"],
  "references": ["Student (1908)", "Welch (1947)"]
}
  
```

4.6 Ordre de construction recommandé

- Étape 1 — Stats Engine seul (Python/FastAPI), sans IA ni Rust : statistiques descriptives, 4 à 5 tests, régression linéaire simple/multiple, testé en ligne de commande.
- Étape 2 — Data Service (Rust) : upload, parsing Polars, retour d'un schéma JSON des colonnes, testé indépendamment.
- Étape 3 — AI Orchestrator (Python) : relie prompt + schéma → plan JSON → appel Stats Engine → réponse rédigée.
- Étape 4 — Gateway (Rust) : unification de l'accès une fois les trois services fonctionnels en local.

- Étape 5 — Viz Service puis Geo Service : couche de restitution graphique et cartographique, construite après la fiabilisation du moteur d'analyse.

5. Catalogue mathématique, probabiliste et statistique complet

Cette section constitue le référentiel scientifique de la plateforme. Chaque bloc doit, dans le Stats Engine, être implémenté comme une ou plusieurs fonctions documentées selon le standard défini en section 4.5 : entrées, sorties, hypothèses, méthode numérique, limites, références. Les bibliothèques de référence pour l'implémentation sont indiquées à titre indicatif (NumPy, SciPy, statsmodels, scikit-learn).

5.1 Fondations algébriques et analytiques

Socle mathématique nécessaire au bon fonctionnement du moteur numérique, notamment pour l'implémentation des méthodes de résolution et d'optimisation.

Algèbre générale

- Commutativité, associativité, distributivité, éléments neutres et inverses
- Identités remarquables, factorisation, résolution d'équations et d'inéquations
- Systèmes d'équations linéaires, suites arithmétiques et géométriques

Fonctions

- Fonctions affine, polynomiale, rationnelle, exponentielle, logarithmique
- Fonctions trigonométriques et hyperboliques, fonctions composées et multivariées

Calcul différentiel

$$f'(x) = \lim_{h \rightarrow 0} [f(x+h) - f(x)] / h$$

$$(fg)' = f'g + fg' \quad (\text{règle du produit})$$

$$(f/g)' = (f'g - fg') / g^2 \quad (\text{règle du quotient})$$

$$(f \circ g)'(x) = f'(g(x)) \cdot g'(x) \quad (\text{règle de chaîne})$$

$$(e^x)' = e^x \quad (\ln x)' = 1/x \quad (\sin x)' = \cos x \quad (\cos x)' = -\sin x$$

Calcul intégral

$$\int_a^b f(x) dx = F(b) - F(a), \quad \text{avec } F'(x) = f(x)$$

- Intégration par parties, changement de variable, intégrales impropres
- Intégrales multiples, curvilignes, de surface ; intégration numérique

Analyse multivariée

$$\nabla f = [\partial f / \partial x_1, \dots, \partial f / \partial x_n]^T \quad (\text{gradient})$$

$$H_{ij} = \partial^2 f / (\partial x_i \partial x_j) \quad (\text{matrice hessienne})$$

$$\nabla^2 f = \sum_i \partial^2 f / \partial x_i^2 \quad (\text{laplacien})$$

- Divergence, rotationnel, Jacobienne

5.2 Algèbre linéaire

Fondation indispensable pour la régression, la réduction dimensionnelle (PCA), le traitement matriciel des données et le machine learning.

$$A x = b \quad A^{-1} A = I$$

$$A v = \lambda v, \quad \det(A - \lambda I) = 0 \quad (\text{valeurs et vecteurs propres})$$

- $A = Q \Lambda Q^{-1}$ (décomposition spectrale)
 $A = U \Sigma V^T$ (décomposition en valeurs
 singulières – SVD)
- Produit scalaire et vectoriel, norme, distance, projection
 - Déterminant, matrice inverse, rang, trace, transposée
 - Matrices symétriques, orthogonales, définies positives
 - Décompositions LU, QR, Cholesky (nécessaires à la résolution numérique performante)

5.3 Probabilités — fondations

Axiomes de Kolmogorov

$$P(A) \geq 0 \quad P(\Omega) = 1 \quad P(A \cup B) = P(A) + P(B) \quad \text{si } A, B \text{ disjoints}$$

Probabilité conditionnelle et théorème de Bayes

$$P(A|B) = P(A \cap B) / P(B)$$

$$P(A|B) = P(B|A) \cdot P(A) / P(B) \quad (\text{théorème de Bayes})$$

$$P(A \cap B) = P(A) \cdot P(B) \quad (\text{indépendance})$$

Variables aléatoires

$$E[X] = \sum x \cdot p(x) \quad \text{ou} \quad E[X] = \int x \cdot f(x) dx \quad (\text{espérance})$$

$$\text{Var}(X) = E[(X-\mu)^2] = E[X^2] - E[X]^2 \quad (\text{variance})$$

$$\text{Cov}(X, Y) = E[(X-E[X])(Y-E[Y])] \quad (\text{covariance})$$

$$\rho_{XY} = \text{Cov}(X, Y) / (\sigma_X \cdot \sigma_Y) \quad (\text{corrélation})$$

Théorèmes fondamentaux

$$\bar{X}_n \rightarrow \mu \quad (\text{loi des grands nombres})$$

$$(\bar{X} - \mu) / (\sigma/\sqrt{n}) \rightarrow N(0,1) \quad (\text{théorème central limite})$$

5.4 Catalogue des lois de probabilité

Pour chaque loi listée ci-dessous, le moteur doit exposer : fonction de densité/masse (PDF/PMF), fonction de répartition (CDF), moyenne, variance, quantile, génération d'échantillons aléatoires, fonction génératrice des moments.

Lois discrètes

Catégorie	Éléments couverts	Usage cible dans la plateforme
Bernoulli	$P(X=1)=p, P(X=0)=1-p$	Modélisation d'un événement binaire (succès/échec, conversion, défaut)
Binomiale	$P(X=k) = C(n,k) p^k(1-p)^{n-k}$	Nombre de succès sur n essais indépendants
Géométrique / Binomiale négative	Nombre d'essais avant le premier/le k-ième succès	Temps d'attente avant un événement
Hypergéométrique	Tirage sans remise dans une population finie	Échantillonnage, contrôle qualité

Catégorie	Éléments couverts	Usage cible dans la plateforme
Poisson	$P(X=k) = e^{-\lambda} \lambda^k / k!$	Comptage d'événements rares (arrivées, incidents, appels)
Multinomiale	Généralisation de la binomiale à K catégories	Répartition de choix entre plusieurs catégories
Zipf, uniforme discrète	Distributions de rang et d'équiprobabilité	Analyse de fréquences, tirages équiprobables

Lois continues

Catégorie	Éléments couverts	Usage cible dans la plateforme
Normale (Gauss)	$f(x) = (1/\sigma\sqrt{2\pi}) e^{-(x-\mu)^2/2\sigma^2}$	Référence pour tests paramétriques, intervalles de confiance
Uniforme continue	Densité constante sur [a,b]	Simulation, génération aléatoire de référence
Exponentielle	$f(x) = \lambda e^{-(\lambda x)}$	Durées de vie sans mémoire, délais entre événements Poisson
Gamma / Bêta	Généralisation de l'exponentielle / distribution sur [0,1]	Modélisation de durées, proportions, priors bayésiens
Weibull	Modélisation de durée de vie à risque variable	Fiabilité, analyse de survie
Log-normale	$\log(X)$ suit une loi normale	Revenus, tailles de villes, variables strictement positives asymétriques
Student (t)	Approxime la normale pour petits échantillons	Tests t, intervalles de confiance à variance inconnue
Chi-carré (χ^2)	Somme de carrés de normales centrées réduites	Tests d'indépendance, d'ajustement, variance
Fisher (F)	Rapport de deux variances (χ^2 normalisés)	ANOVA, comparaison de variances, régression
Pareto, Cauchy, Logistique, Laplace, Rayleigh, Gumbel, Dirichlet	Distributions spécialisées (queues lourdes, extrêmes, compositionnelles)	Modélisation de risques extrêmes, valeurs extrêmes, données compositionnelles

5.5 Statistiques descriptives

- Tendence centrale : moyenne arithmétique, géométrique, harmonique, médiane, mode
- Dispersion : variance, écart-type, coefficient de variation, étendue, écart interquartile (IQR), écart absolu médian (MAD)
- Position : quantiles, percentiles, quartiles
- Forme de la distribution : asymétrie (skewness), aplatissement (kurtosis)
- Association bivariable : covariance, corrélation de Pearson, de Spearman, de Kendall

5.6 Échantillonnage et méthodes de rééchantillonnage

- Échantillonnage aléatoire simple, stratifié, systématique, en grappes (cluster)
- Bootstrap, jackknife, tests de permutation, simulation de Monte-Carlo

$\theta_1^*, \theta_2^*, \dots, \theta_B^* \rightarrow$ estimation de la distribution de $\hat{\theta}$ par ré-échantillonnage

5.7 Estimation statistique

$L(\theta|x) = \prod_i f(x_i|\theta)$ (vraisemblance)
 $\ell(\theta) = \log L(\theta)$ (log-vraisemblance)
 $\hat{\theta} = \operatorname{argmax}_{\theta} L(\theta)$ (estimateur du maximum de vraisemblance)
 $\hat{\theta}_{\text{MAP}} = \operatorname{argmax}_{\theta} P(\theta|D)$ (estimateur du maximum a posteriori)

Intervalles de confiance

$\bar{x} \pm z_{(\alpha/2)} \cdot \sigma/\sqrt{n}$ (variance connue)
 $\bar{x} \pm t_{(\alpha/2, n-1)} \cdot s/\sqrt{n}$ (variance inconnue)

5.8 Tests d'hypothèses

Pour chaque test, le moteur documente H_0 , H_1 , la statistique de test, la p-value, le seuil α retenu, la puissance, ainsi que les risques d'erreur de première et deuxième espèce.

Catégorie	Éléments couverts	Usage cible dans la plateforme
Comparaison de 2 groupes	Test t (Student, Welch), test z, Mann-Whitney (Wilcoxon rank-sum), test des signes	Comparer une moyenne entre deux groupes indépendants ou appariés
Comparaison de plus de 2 groupes	ANOVA à un facteur, ANOVA à deux facteurs, MANOVA, ANCOVA, Kruskal-Wallis, Friedman	Comparer des moyennes entre plusieurs groupes, avec ou sans covariables
Association / indépendance	Chi-carré d'indépendance, test exact de Fisher, McNemar	Tester la relation entre deux variables catégorielles
Normalité et forme	Shapiro-Wilk, Kolmogorov-Smirnov, Anderson-Darling, Jarque-Bera	Vérifier les conditions d'application des tests paramétriques
Homogénéité des variances	Levene, Bartlett	Condition préalable à l'ANOVA et au test t classique
Séries appariées / non paramétrique	Wilcoxon signed-rank, test des signes	Comparaisons robustes sans hypothèse de normalité

5.9 Régression et modélisation

$y = X\beta + \varepsilon$ (modèle linéaire général)
 $\hat{\beta} = (X^T X)^{-1} X^T y$ (estimateur des moindres carrés ordinaires – OLS)
 $R^2 = 1 - SS_{\text{res}} / SS_{\text{tot}}$ (coefficient de détermination)
 $P(Y=1|X) = 1 / (1 + e^{-(X\beta)})$ (régression logistique)
 $\min_{\beta} [\|y - X\beta\|^2 + \lambda \|\beta\|^2]$ (régression Ridge)

$\min_{\beta} [\|y - X\beta\|^2 + \lambda \|\beta\|_1]$ (régression Lasso)

- Régression polynomiale, Elastic Net, Poisson, binomiale négative, quantile
- Diagnostics associés : multicolinéarité (VIF), hétéroscédasticité (Breusch-Pagan), normalité des résidus, distance de Cook (observations influentes)

5.10 Analyse de variance (ANOVA)

$F = MS_{\text{between}} / MS_{\text{within}}$

- ANOVA à un facteur, à deux facteurs, à mesures répétées, MANOVA, ANCOVA

5.11 Séries temporelles

$X_t = c + \sum_i \phi_i \cdot X_{t-i} + \varepsilon_t$ (modèle autorégressif AR(p))

- Moyenne mobile (MA), ARMA, ARIMA, SARIMA, VAR, GARCH
- Décomposition saisonnière, autocorrélation (ACF), autocorrélation partielle (PACF)
- Filtre de Kalman, modèles à espace d'états

5.12 Analyse multivariée et réduction de dimension

$\text{Cov}(X) = (1/n) X^T X \rightarrow C v = \lambda v$ (analyse en composantes principales – PCA)

- Analyse en composantes indépendantes (ICA), analyse factorielle
- Analyse discriminante, corrélation canonique, positionnement multidimensionnel (MDS), analyse des correspondances

5.13 Classification non supervisée (clustering)

$\min \sum_k \sum_{(x_i \in C_k)} \|x_i - \mu_k\|^2$ (k-means)

- K-means++, classification hiérarchique, DBSCAN, HDBSCAN
- Modèles de mélange gaussien (GMM), clustering spectral

5.14 Optimisation et méthodes numériques

$\theta_{(t+1)} = \theta_t - \eta \nabla f(\theta_t)$ (descente de gradient)

- Newton-Raphson, quasi-Newton (BFGS, L-BFGS), descente de coordonnées, gradient stochastique
- Optimisation sous contraintes, programmation linéaire et quadratique, optimisation convexe
- Interpolation, différences finies, recherche de racines, décompositions LU/QR/Cholesky/SVD

5.15 Statistiques bayésiennes

$P(\theta|D) \propto P(D|\theta) \cdot P(\theta)$ (théorème de Bayes appliqué à l'inférence)

- Priors conjugués, MCMC, algorithme de Metropolis-Hastings, échantillonnage de Gibbs
- Monte-Carlo hamiltonien (HMC), NUTS, régression bayésienne, réseaux bayésiens

5.16 Analyse de survie

- Estimateur de Kaplan-Meier, modèle de Cox à risques proportionnels

- Fonctions de risque, de survie, censure à droite/gauche

5.17 Inférence causale

- Graphes causaux dirigés acycliques (DAG), facteurs de confusion
- Score de propension, appariement (matching), pondération par probabilité inverse
- Variables instrumentales, différence de différences, régression sur discontinuité, forêts causales, analyse de médiation

5.18 Machine learning (modules avancés)

- Régression linéaire et logistique, Naive Bayes, k plus proches voisins (kNN)
- Machines à vecteurs de support (SVM), arbres de décision, forêts aléatoires
- Gradient boosting, XGBoost, réseaux de neurones, détection d'anomalies

5.19 Théorie de l'information

$$H(X) = -\sum_x P(x) \log P(x) \quad (\text{entropie de Shannon})$$

$$I(X;Y) = \sum_x \sum_y P(x,y) \log[P(x,y) / (P(x)P(y))] \quad (\text{information mutuelle})$$

$$D_{\text{KL}}(P\|Q) = \sum_x P(x) \log[P(x)/Q(x)] \quad (\text{divergence de Kullback-Leibler})$$

- Entropie croisée, divergence de Jensen-Shannon, entropie conditionnelle

5.20 Analyse A/B et test de puissance

- Conception et analyse de tests A/B, calcul de la taille d'échantillon nécessaire
- Analyse de puissance statistique (power analysis), taille d'effet (Cohen's d, eta carré)

6. Module optionnel — lois et formalisme de la physique quantique

Ce module n'est pas nécessaire au fonctionnement de STATIA en version 1. Il est documenté ici à titre prospectif, pour un futur module de calcul quantique / apprentissage automatique quantique (« Quantum ML »), qui pourrait apporter un avantage de calcul sur certains problèmes d'optimisation ou d'échantillonnage à très grande échelle.

6.1 Pourquoi ce module est distinct du cœur statistique

Le formalisme quantique repose sur l'algèbre linéaire dans des espaces de Hilbert complexes et sur une notion de probabilité fondamentalement différente de la théorie classique des probabilités utilisée en section 5. Il ne remplace ni les tests statistiques, ni la régression, ni le machine learning classique : il constitue une piste d'évolution à long terme, pertinente uniquement si la plateforme souhaite un jour proposer du calcul quantique simulé ou des algorithmes hybrides quantique-classique.

6.2 Équations fondamentales

$i\hbar \partial/\partial t \psi\rangle = \hat{H} \psi\rangle$ dépendante du temps)	(équation de Schrödinger
$\hat{H} \psi = E \psi$ stationnaire)	(équation de Schrödinger
$\Delta x \cdot \Delta p \geq \hbar/2$ Heisenberg)	(principe d'incertitude de
$[\hat{x}, \hat{p}] = i\hbar$ canonique)	(relation de commutation
$ \psi(x,t) ^2$ présence)	(densité de probabilité de
$\langle A \rangle = \langle \psi \hat{A} \psi \rangle$ opérateur)	(valeur moyenne d'un

6.3 États, mesure et information quantique

$ \psi\rangle = \alpha 0\rangle + \beta 1\rangle$, $ \alpha ^2 + \beta ^2 = 1$	(qubit)
$\rho(t) = U \rho(0) U^\dagger$ densité)	(évolution d'une matrice de
$S(\rho) = -\text{Tr}(\rho \log \rho)$	(entropie de von Neumann)
• Matrices de densité, états intriqués, mesure projective • Portes quantiques (Hadamard, CNOT, Pauli X/Y/Z) — briques de base d'un circuit quantique simulé	

6.4 Pistes d'application future, si le module est activé

- Recuit quantique simulé pour des problèmes d'optimisation combinatoire (allocation de ressources, planification)
- Algorithmes d'échantillonnage quantique pour accélérer certaines étapes de Monte-Carlo à très grande dimension

- Exploration de l'apprentissage automatique quantique (Quantum ML) sur des jeux de données à très haute dimensionnalité

Recommandation : ne pas engager ce module avant que le cœur statistique (section 5) et la couche de visualisation (section 7) soient stabilisés et adoptés par les utilisateurs. Ce module reste une note de veille technologique, pas un engagement de développement.

7. Module de visualisation et de tableaux de bord (Power BI-like)

7.1 Objectif du module

Ce module constitue la couche de restitution visuelle de la plateforme. Il doit permettre à l'utilisateur de passer d'un résultat d'analyse ponctuel à un tableau de bord vivant, partageable et actualisable, comparable en usage à Power BI ou Tableau, tout en restant piloté par le langage naturel pour sa création initiale.

7.2 Composants fonctionnels

Catégorie	Éléments couverts	Usage cible dans la plateforme
Bibliothèque de visuels	Histogramme, boîte à moustaches, nuage de points, courbe, aires empilées, diagramme circulaire, treemap, jauge, carte de chaleur	Représentation adaptée à chaque type de variable et de résultat statistique
Cartographie ciblée	Cartes choroplèthes par découpage administratif, cartes de points géolocalisés, cartes de densité, agrégation par zone personnalisée	Analyse territoriale (ventes par région, couverture santé, répartition démographique)
Tableaux de bord interactifs	Glisser-déposer de widgets, filtres croisés, segmentation dynamique (« slicers »), drill-down hiérarchique	Exploration libre des données par l'utilisateur final
Cartes de indicateurs (KPI)	Cartes numériques avec comparaison à une période de référence et code couleur d'alerte	Suivi de performance en un coup d'œil
Export et partage	PDF, PNG, XLSX, PowerPoint ; lien de partage sécurisé avec gestion des droits	Diffusion des résultats à des tiers non utilisateurs de la plateforme
Actualisation des données	Rafraîchissement programmé ou déclenché à l'import d'un nouveau fichier	Maintien à jour des tableaux de bord sans reconstruction manuelle

7.3 Flux de création d'un tableau de bord

- L'utilisateur formule une demande de haut niveau (« crée un tableau de bord de suivi des ventes par région et par mois »).
- Le système propose une disposition initiale de widgets pertinents, générée automatiquement à partir du schéma des données.
- L'utilisateur ajuste par glisser-déposer, complète par de nouvelles demandes en langage naturel, ou édite manuellement chaque visuel.
- Le tableau de bord est sauvegardé, versionné et peut être dupliqué comme modèle pour d'autres jeux de données similaires.

7.4 Spécificités cartographiques pour le contexte ouest-africain

- Support natif des découpages administratifs du Togo et de l'espace UEMOA (régions, préfectures, communes, cantons).
- Import de fonds de carte personnalisés (GeoJSON/Shapefile) pour des découpages non standards (zones de projet, bassins versants, zones sanitaires).

- Gestion des cas de données incomplètes ou peu granulaires, fréquents sur les données de terrain en Afrique de l'Ouest.

7.5 Séquencement de construction

Ce module est développé après la fiabilisation complète du moteur d'analyse (sections 4 et 5). Ordre recommandé : (1) rendu de graphiques simples liés directement aux résultats d'une analyse ponctuelle ; (2) assemblage de plusieurs visuels en tableau de bord statique ; (3) interactivité et filtres croisés ; (4) cartographie ; (5) partage, export et actualisation programmée.

8. Exigences non fonctionnelles

8.1 Fiabilité et exactitude

- Aucun résultat chiffré ne doit provenir directement d'une génération du modèle de langage : toute valeur numérique affichée doit être traçable jusqu'à un appel réel du Stats Engine.
- Couverture de tests unitaires exhaustive sur chaque fonction du registre de formules, avec jeux de données de référence à résultats connus (validation croisée avec R/SciPy).

8.2 Performance et scalabilité

- Traitement de fichiers jusqu'à plusieurs centaines de milliers de lignes sans dégradation perceptible de l'expérience utilisateur.
- Architecture microservices permettant la montée en charge indépendante de chaque composant (Data Service, Stats Engine, Viz Service).

8.3 Sécurité et confidentialité des données

- Chiffrement des données au repos et en transit ; isolation stricte des données entre utilisateurs/organisations.
- Politique de rétention et de suppression des fichiers importés configurable par l'utilisateur.

8.4 Accessibilité et localisation

- Interface disponible en français en priorité, avec support anglais ; terminologie statistique bilingue.
- Compatibilité avec des connexions à bande passante limitée, contexte pertinent pour l'Afrique de l'Ouest.

8.5 Explicabilité

- Chaque résultat est accompagné d'une explication en langage naturel et d'un accès au détail méthodologique complet (« Show me why »).

9. Feuille de route de réalisation

Catégorie	Éléments couverts	Usage cible dans la plateforme
Phase 0 — Fondations	Mise en place du dépôt, conventions, structure microservices, environnement de développement	Rust + Python
Phase 1 — Stats Engine minimal	Statistiques descriptives, 5 tests d'hypothèses, régression linéaire simple/multiple, vérification d'hypothèses de base	Python/FastAPI, SciPy, statsmodels
Phase 2 — Data Service	Upload, parsing CSV/XLSX, typage automatique, nettoyage de base, retour de schéma	Rust, Polars/Arrow
Phase 3 — AI Orchestrator	Compréhension du prompt, plan JSON, appel du Stats Engine, boucle de vérification, rédaction de l'explication	Python/FastAPI, API LLM
Phase 4 — Gateway & intégration	Unification des services, authentification, montée en charge des uploads	Rust/axum
Phase 5 — Viz Service	Génération de graphiques liés aux résultats, premiers tableaux de bord statiques	Python, Vega-Lite/Plotly
Phase 6 — Interactivité & cartographie	Filtres croisés, segmentation, cartographie ciblée, export/partage	React/Next.js, Geo Service
Phase 7 — Extension du catalogue scientifique	Séries temporelles, bayésien, causal, machine learning avancé	Python, extension du registre de formules
Phase 8 — Module quantique (optionnel)	Veille et prototypage si pertinence confirmée	À réévaluer post-Phase 7

10. Annexes

10.1 Glossaire synthétique

- p-value : probabilité d'observer un résultat au moins aussi extrême que celui obtenu, sous l'hypothèse nulle.
- Intervalle de confiance : plage de valeurs dans laquelle le paramètre estimé se situe avec un niveau de confiance donné (généralement 95 %).
- Hétéroscédasticité : variance non constante des résidus d'un modèle de régression, invalidant certaines hypothèses classiques.
- Multicolinéarité : forte corrélation entre variables explicatives d'un modèle, pouvant fausser l'interprétation des coefficients.
- Bootstrap : technique de ré-échantillonnage avec remise permettant d'estimer la distribution d'un estimateur sans hypothèse théorique forte.

10.2 Références bibliographiques indicatives

- Casella, G. & Berger, R. L. — Statistical Inference
- Hastie, T., Tibshirani, R. & Friedman, J. — The Elements of Statistical Learning
- Gelman, A. et al. — Bayesian Data Analysis
- Documentation officielle SciPy, statsmodels, scikit-learn, Polars

10.3 Prochaines étapes immédiates suggérées

- Valider le périmètre exact de la Phase 1 (liste fermée des 5 tests et des modèles de régression couverts).
- Rédiger les premières fiches de méthode selon le standard de la section 4.5 pour les fonctions de la Phase 1.
- Mettre en place le squelette du Stats Engine (FastAPI) avec un premier endpoint fonctionnel testé de bout en bout.